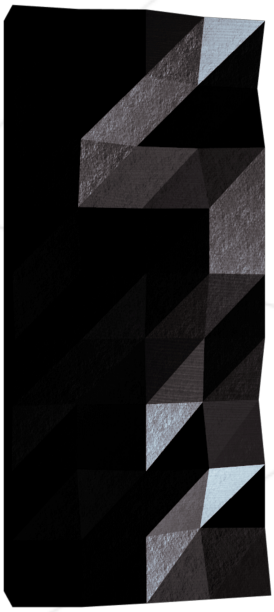


H42N42

Simulate the life of a population of creatures threatened by a terrifying virus

Summary: This project aims to help you discover the basics of web programming on the client side in `OCaml`*, thanks to Ocsigen.*

Version: 6.0

Contents

Chapter I

Foreword

Since its creation, the WWW has evolved significantly. In the beginning, it consisted only of static pages (HTML documents linked together). These pages were quickly supplemented by pages generated on the server side from databases (Wikipedia, online stores, online newspapers, forums...). More recently, the WWW has undergone a radical transformation, becoming a platform capable of running real applications (word processors, games...) whose calculations take place partly on the server and partly in the browser.

The 90's web languages (`PHP`, `JavaScript`...) were not designed for this kind of application. Most of them are script-based languages that offer no proper static guarantees (no static typing), making debugging tedious, code maintenance difficult, and leaving the system open to many security breaches.

Moreover, web developers often have to work with several languages: `JavaScript` on the client side, `PHP`, `Ruby`, or `Python` – to name just a few – on the server side, `SQL` for databases, etc. They must convert data from one language to another and rewrite functions multiple times if they want to use them on both sides, which creates new hazards and introduces additional security problems.

The research project `Ocsigen` aims to create a new way to program modern web applications, focusing on *expressiveness* and *reliability*. The project has been successful thanks to the creation of a complete framework, distributed as open source software, containing a compiler, a web server, and many libraries. `Ocsigen` is based on several high-level concepts found in the `OCaml` language (polymorphic variants, closures, generalized algebraic types, etc.) to write complex applications with very few lines of code. Most importantly, `Ocsigen` takes advantage of powerful type systems like `OCaml`'s to check many properties of the program during compilation. It can even verify at compile time that your programs will not generate pages that violate `W3C` recommendations or whose forms do not match the services they target.

Most importantly, `Ocsigen` allows you to program the entire application (server and client) in `OCaml` within the same codebase. During compilation, the parts that should be executed by the browser are extracted and converted into highly efficient `JavaScript` by the `Ocsigen Js_of_ocaml` compiler.

`Ocsigen` is developed by the `CNRS`, University of Paris Diderot (P7), and `Inria` (French National Institute for Research in Computer Science and Automation) research labs, as well as by the company BeSport, which develops a sports-focused social network.

`Ocsigen` is already used by distinguished organizations worldwide, such as the University of New York (CSGB Genomics Core), startups, open source projects, and research laboratories. `Meta`, for instance, uses the `Js_of_ocaml` compiler to execute their `OCaml` code in browsers.

`Ocsigen` follows a general trend aimed at improving web programming techniques through static checking. For instance, some projects attempt to add static typing to existing languages, such as `Microsoft TypeScript` and `Facebook Flow` for `JavaScript`, `Facebook Hack` for `PHP`, or `Facebook Infer` for `C`, `C++`, and `Java`. Other recent frameworks increasingly adopt strongly-typed languages, such as `OCaml`, `Scala` (used by `X`), or `Haskell`.

ocsigen

be sport

université PARIS DIDEROT PARIS 7

Inria INVENTEURS DU MONDE NUMÉRIQUE

cnrs CENTRE NATIONAL DE LA RECHERCHE SCIENTIFIQUE

Chapter II

Introduction

For millennia, Creatures have lived in a peaceful land bordered by a river. Unfortunately, the river has been polluted by H42N42, a deadly and highly infectious virus. The Creatures will not survive without your assistance! Your goal is to help them stay away from the river and bring the sick ones to the hospital where they will be healed so they don't contaminate the others.

In this project, you will write an interactive simulator of this Creature world with a Web UI written in `OCaml` on the client side.

Chapter III

Objectives

`Ocsigen` allows you to manage every aspect of Web app programming, and to create client-server websites and applications. For a gentle start, this project will help you discover client-side programming with the library Eliom and the compiler Js_of_ocaml, and will teach you to:

1. Run an `OCaml` program in a web browser using the `Ocsigen` `Js_of_ocaml` compiler;
2. Statically validate the `HTML` pages you generate to prevent your program from creating invalid pages;
3. Interact with the browser's DOM and call `Javascript` methods using your `OCaml` program;
4. Program handlers for mouse events;
5. Program cooperative threads in monadic style with `Ocsigen` `Lwt`.

Chapter IV

General instructions

The project must be entirely programmed using the latest version of `OCaml`, the `Js_of_ocaml` compiler, and the `Eliom` library from the `Ocsigen` project.

You will find the Ocsigen documentation on the project website. Take time to read the tutorials before starting.

IV.1 Technical Requirements

- Each Creature must be commanded by a `Lwt` thread.
- It will be displayed as an HTML element (a `<div>` or `<img>` for instance). All dynamically generated elements of the page must be created using the `TyXML` library (such as `Eliom_content.Html.D`).
- Mouse events must be programmed with the `Lwt_js_event` module of `Js_of_ocaml`. You will find examples of this module in tutorials on the `Ocsigen` website.
- Your project must use `OCaml` version 4.14 or higher.
- You must use `Ocsigen/Eliom` version 10.0 or higher and `Js_of_ocaml` version 5.0 or higher.
- Your project must include a `Makefile` with at least the following rules: `all`, `clean`, `fclean`, and `re`.
- The `Makefile` must not recompile unnecessarily (use proper dependencies).

IV.2 Project Organization

Your project must follow a clear and logical structure. At minimum, you should have:

- A `src/` directory containing your `.eliom` source files
- A `static/` directory for static assets (CSS, images, etc.)

- Configuration files at the root (`.conf.in` files for Ocsigen server)
- A `Makefile` for building the project
- Docker configuration files (see next chapter)

IV.3 Advice

- Keep your browser debugging tools handy;
- Read the `Ocsigen tutorials`;
- Start with a simple working version before adding complexity;
- Test frequently in the browser during development;
- Use the browser console to debug JavaScript compilation issues.

Chapter V

Docker Setup and Deployment

To ensure your project can be easily deployed and tested in any environment, you must containerize your application using `Docker`.

V.1 Docker Requirements

Your project must include the following Docker configuration files:

- A `Dockerfile` that:
  - Uses an appropriate base image with OCaml support (e.g., `ocaml/opam`)
  - Installs all necessary dependencies (Ocsigen, Eliom, Js_of_ocaml, etc.)
  - Copies your source code into the container
  - Compiles your application
  - Exposes the appropriate port (default: 8080)
  - Defines the command to start the Ocsigen server
- A `docker-compose.yml` file that:
  - Defines the service for your application
  - Maps the container port to the host (e.g., 8080:8080)
  - Sets up any necessary volumes for development
  - Configures environment variables if needed

V.2 Deployment Instructions

Your application must be launchable with a single command:

```
docker-compose up --build
```

After running this command, the application should be accessible in a web browser at `http://localhost:8080` (or the port you specified).

The Docker setup must:

- Build successfully without manual intervention
- Install all dependencies automatically
- Compile the OCaml code to JavaScript using Js_of_ocaml
- Start the Ocsigen server automatically
- Serve the application on the specified port
- Handle server restarts gracefully during development (optional but recommended)

Your project will not be evaluated if it cannot be launched using Docker. Make sure to test your Docker configuration on a clean system before submission.

Chapter VI

Mandatory part

VI.1 Game Overview

You will create an interactive browser-based game where creatures called **Creets** move around in a rectangular play area. The game features:

- A **toxic river** at the top of the screen that contaminates Creets
- A **hospital** at the bottom where sick Creets can be healed
- **Player interaction** through mouse drag-and-drop to save Creets
- **Progressive difficulty** as the game advances

VI.1.1 Game Objective

The goal is to keep the Creet population alive as long as possible by:

- Preventing healthy Creets from touching the toxic river
- Dragging sick Creets to the hospital for healing
- Managing the spread of contamination

The game ends when all Creets are either dead or contaminated with no healthy Creets remaining.

VI.2 Game Mechanics

VI.2.1 Creet Movement and Behavior

- **Movement Pattern:** Creets move in straight lines at constant speed. They may randomly change direction, but not too frequently (allow players to predict paths while maintaining unpredictability).
- **Boundary Behavior:** When a Creet reaches any edge of the game area, it must bounce back realistically (angle of incidence equals angle of reflection).

- **Initial Population:** Start with a reasonable number of Creets (e.g., 5-10) based on their size. The display should be clear and not cluttered.
- **Reproduction:** As long as at least one healthy Creet exists, new Creets spawn periodically (e.g., every few seconds). Healthy Creets never die of old age.
- **No Collisions:** Creets pass through each other without bouncing. Only contamination occurs on contact.

VI.2.2 Player Interaction

- **Drag and Drop:** The player can click and drag any Creet to move it anywhere on the screen using the mouse.
- **Invulnerability While Dragging:** A Creet being dragged by the player cannot be contaminated. This allows safe transport of healthy Creets away from danger.
- **Dropping Creets:** Creets can be dropped anywhere, including outside the normal game boundaries. They will resume normal movement from their new position.

VI.2.3 Contamination System

- **River Contamination:** Any Creet that touches the toxic river at the top of the screen immediately becomes sick.
- **Visual Change:** Contaminated Creets instantly change color to distinguish them from healthy ones.
- **Speed Reduction:** Sick Creets move 15% slower than healthy ones.
- **Spread Mechanism:** When a contaminated Creet touches a healthy Creet, there is a 2% chance of transmission per contact/iteration. Check this probability continuously while they are in contact.
- **Contact Detection:** Use distance calculation between Creet centers and their radii to determine if they are touching. Be precise: no contamination without actual contact.

VI.2.4 Hospital Healing

- **Hospital Location:** The hospital is located at the bottom of the game area.
- **Healing Requirement:** Only Creets that are **manually dragged and dropped** by the player into the hospital area will be healed. Sick Creets that randomly wander into the hospital are NOT healed.
- **Healing Effect:** Healed Creets return to healthy status: they regain their original color, return to normal speed, and are no longer contagious.
- **Restrictions:** Berserk and mean Creets (see below) cannot be healed and cannot be dragged to the hospital.

VI.2.5 Special Creet Types

When a Creet becomes contaminated, it may evolve into one of two dangerous types:

Berserk Creets

- **Probability:** 10% chance when a Creet gets sick (check once every 10 seconds after contamination).
- **Visual:** Different color from regular sick Creets.
- **Growth:** Diameter increases by 10% every 10 seconds.
- **Death:** Dies when size reaches 4× original diameter (approximately 2 minutes 30 seconds after becoming berserk).
- **Danger:** Larger size means higher chance of touching and contaminating other Creets.
- **Restrictions:** Cannot be grabbed by the player or healed.

Mean Creets

- **Probability:** 10% chance when a Creet gets sick (check once every 10 seconds after contamination).
- **Visual:** Unique color different from both regular sick and berserk Creets.
- **Size:** Shrinks to 85% of original size (15% smaller).
- **Behavior:** Actively chases healthy Creets to contaminate them. Calculate direction toward nearest healthy Creet and move toward it.
- **Death:** Dies after exactly 1 minute of being mean.
- **Restrictions:** Cannot be grabbed by the player or healed.

Important Notes

- A contaminated Creet can become either berserk OR mean, but NOT both.
- Once a Creet becomes berserk or mean, it remains that type until death.
- Regular sick Creets (not berserk or mean) can still be healed at the hospital.

VI.2.6 Difficulty Progression

- **Speed Increase:** All Creets gradually accelerate over time, making the game progressively harder.
- **Balancing:** You are free to adjust the rate of acceleration to create a good difficulty curve. The game should:

  - Start easy, giving players time to learn mechanics
  - Gradually become more challenging
  - Eventually become overwhelming, leading to inevitable defeat

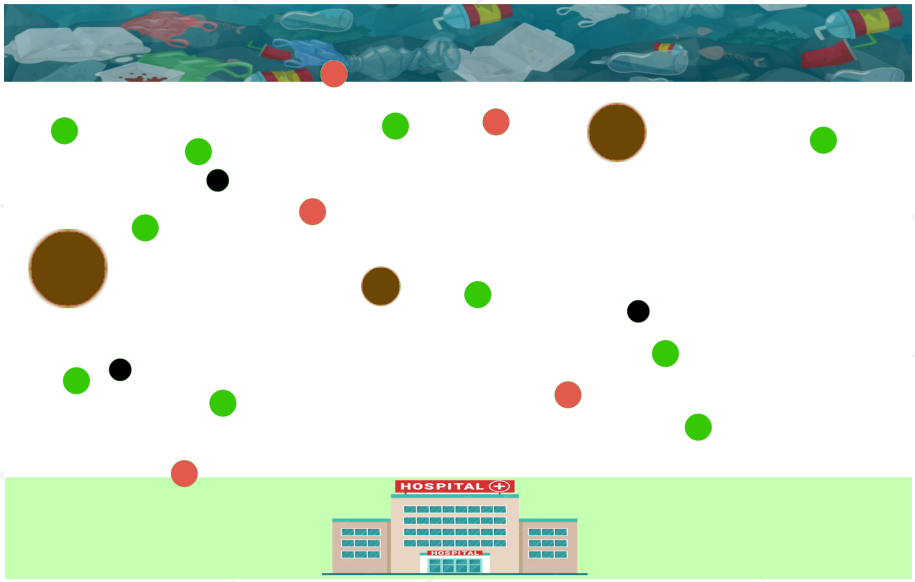
- **Probability Adjustments:** You may modify the specified probabilities (contamination chance, berserk/mean chance) ONLY if necessary for game balance, without fundamentally changing the game's nature.

VI.2.7 Game Over Condition

- The game ends when there are no healthy or living Creets remaining.
- Display a clear "GAME OVER" message when this condition is met.
- The message should be visible and unambiguous to the player.

VI.2.8 Implementation Guidelines

- **Visual Clarity:** Choose Creet sizes that make the game playable and visually clear. Very small Creets would be hard to interact with; very large ones would clutter the screen.
- **Color Scheme:** Use distinct colors for:
  - Healthy Creets
  - Regular sick Creets
  - Berserk Creets
  - Mean Creets
- **Performance:** Ensure smooth animation even with multiple Creets on screen. Each Creet should run in its own Lwt thread.
- **User Feedback:** Make it clear when a Creet is being dragged, when contamination occurs, and when healing happens.


HOSPITAL

Chapter VII

Bonus part

When you have committed yourself to a project and are happy with the result, it is only natural to want to keep going! Of course, the bonus part will only count if the mandatory part has been thoroughly completed.

The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been integrally done and works without malfunctioning. If you have not passed ALL the mandatory requirements, your bonus part will not be evaluated at all.

VII.1 Bonus 1: Advanced Graphics & Visual Effects

Enhance the visual appeal of your game with professional graphics and animations:

- Use custom sprites or images for different Creet types (healthy, sick, berserk, mean) instead of simple colored shapes
- Implement smooth CSS animations or transitions for state changes (e.g., when a Creet gets contaminated)
- Add visual effects such as:
  - Particle effects when Creets get contaminated
  - Glow or pulse effects for berserk Creets
  - Shadow effects for depth perception
  - Animated background (e.g., flowing river)
- Create a cohesive visual theme with consistent styling across all game elements
- Implement responsive design that adapts to different screen sizes

VII.2 Bonus 2: Interactive Control Panel

Implement a comprehensive control panel that allows players to customize the game experience in real-time:

- Add sliders to adjust game parameters such as:
  - Creet movement speed
  - Contamination probability
  - Reproduction rate
  - Number of initial Creets
  - Berserk/mean Creet probability
- Include buttons to:
  - Pause/resume the game
  - Reset the game
  - Toggle different game modes or difficulty levels
- Display real-time information:
  - Current number of healthy/sick/berserk/mean Creets
  - Game duration
  - Creets saved at the hospital
- The control panel should be well-designed, intuitive, and not obstruct the game area
- All parameter changes must take effect immediately without requiring a game restart

VII.3 Bonus 3: Sound Effects & Audio

Add an audio dimension to enhance the gaming experience:

- Background music that loops continuously (with volume control)
- Sound effects for key events:
  - Creet contamination
  - Creet healing at the hospital
  - Creet death (berserk/mean)
  - Game over
  - Creet reproduction

- Audio controls allowing players to:
  - Mute/unmute all sounds
  - Adjust volume levels separately for music and sound effects
  - Toggle individual sound categories
- Ensure audio files are properly loaded and don't cause performance issues
- Audio should enhance the experience without being annoying or overwhelming

VII.4 Bonus 4: Statistics & Scoring System

Implement a comprehensive statistics and scoring system to track player performance:

- Track and display statistics such as:
  - Total survival time
  - Number of Creets saved at the hospital
  - Number of Creets lost to contamination
  - Maximum number of Creets alive simultaneously
  - Number of berserk/mean Creets encountered
- Calculate a score based on:
  - Survival duration
  - Creets saved
  - Efficiency (ratio of saved vs. lost Creets)
- Display a statistics dashboard that can be toggled on/off
- Show a detailed game over screen with final statistics and score
- Optionally: Implement a local leaderboard (using localStorage) to track high scores across sessions
- Present statistics in a clear, visually appealing format (charts, graphs, or styled tables)

VII.5 Bonus 5: Optimized Collision Detection

Implement an efficient spatial data structure to optimize collision detection and improve performance:

- Use a spatial partitioning algorithm such as:
  - Quadtree

  - Grid-based spatial hashing
  - Or another appropriate spatial data structure

- The optimization should significantly reduce the computational complexity of collision detection from $O(n^2)$ to approximately $O(n \log n)$ or better
- Demonstrate the performance improvement:
  - The game should handle a significantly larger number of Creets (e.g., 100+) without performance degradation
  - Provide a way to toggle between optimized and naive collision detection to show the difference
  - Display FPS (frames per second) counter to demonstrate performance
- The implementation must be correct and not introduce bugs in collision detection
- Code should be well-documented explaining the algorithm and data structure used

Chapter VIII

Deliverables

Your repository must contain all the necessary files to build, run, and understand your project. The following files are mandatory:

VIII.1 Required Files

- `README.md`: A comprehensive documentation file that includes:
  - Project description and objectives
  - Prerequisites (Docker, docker-compose)
  - Installation and setup instructions
  - How to run the application (docker-compose command)
  - How to access the application in the browser
  - Brief explanation of the game rules
  - Any additional features or bonuses implemented
- `Dockerfile`: Your Docker configuration for building the application
- `docker-compose.yml`: Docker Compose configuration for easy deployment
- `Makefile`: Build automation with rules: `all`, `clean`, `fclean`, `re`
- `src/`: Directory containing your `.eliom` source files
- `static/`: Directory containing CSS, images, and other static assets
- Configuration files: `.conf.in` files for Ocsigen server configuration

VIII.2 Optional but Recommended

- `.gitignore`: To exclude build artifacts and temporary files
- `.dockerignore`: To optimize Docker build context

- Additional documentation for complex features
- Screenshots or GIFs demonstrating the game in action

Make sure your repository is clean and well-organized. Remove any unnecessary files, commented-out code, or debug outputs before submission.

Chapter IX

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.

IX.1 Before Submission

Before submitting your project, verify that:

- Your Docker setup works on a clean system (test in a fresh environment)
- All mandatory requirements are implemented and functional
- Your README.md is complete and accurate
- Your code is clean, well-organized, and properly commented
- The application runs without errors or warnings
- All files are committed and pushed to your repository